

Отчет по “SmartCity IoT “

<https://github.com/paolalenti/smart-city-iot>

Быков Кирилл 11-311, Гилязов Ренат 11-311, Киселев Егор 11-311

1. Техническое задание

1. Наименование и область применения

SmartCity IoT - система распределенного мониторинга и автоматизации IoT-устройств. Предназначена для высоконагруженного сбора телеметрии и оперативного реагирования на инциденты.

2. Цели и задачи создания системы

- Централизованный учет датчиков и управление их жизненным циклом.
- Обеспечение отказоустойчивого приема данных с защитой от перегрузок.
- Автоматизация сценариев реагирования на основе анализа потока данных.

3. Характеристика объектов автоматизации (EDA структура)

Обмен данными между компонентами осуществляется через брокер сообщений Apache Kafka. Основные топики:

- **device_events**: Синхронизация реестра датчиков.
- **telemetry**: Поток сырых данных.
- **alerts**: События обнаруженных аномалий.
- **commands**: Команды управления устройствами.
- **notifications**: Информационные сообщения для внешних систем.

4. Требования к системе

- **Функциональные:**
 - Валидация входящих запросов по локальной базе разрешенных устройств (PostgreSQL в Ingestor).
 - Ограничение частоты запросов (Rate Limiting) через Redis.
 - Хранение истории в InfluxDB (Time-series).
- **По надежности:**
 - Сохранность сообщений при отказе сервисов-потребителей за счет механизмов Kafka.
- **По безопасности:**
 - Изоляция микросервисов внутри Docker-сети.
 - Фильтрация трафика на уровне API Gateway.

5. Приемка

1. **Успешное развертывание:** Все контейнеры в статусе **up** через **docker-compose**.
2. **Успешность выполнения сценария:** Создание датчика в DM -> Отправка данных -> Появление алертов в логах Automation.
3. **Стресс-тест:** Проверка отсечки запросов при превышении лимита (429 ошибка).

2. User Story, Use Cases, NFR

User Story

Как оператор безопасности, я хочу, чтобы система автоматически включала систему полива или вентиляции при получении критических данных с датчика, чтобы предотвратить ущерб без моего прямого участия, при этом уведомив меня о совершенных действиях и ЧП.

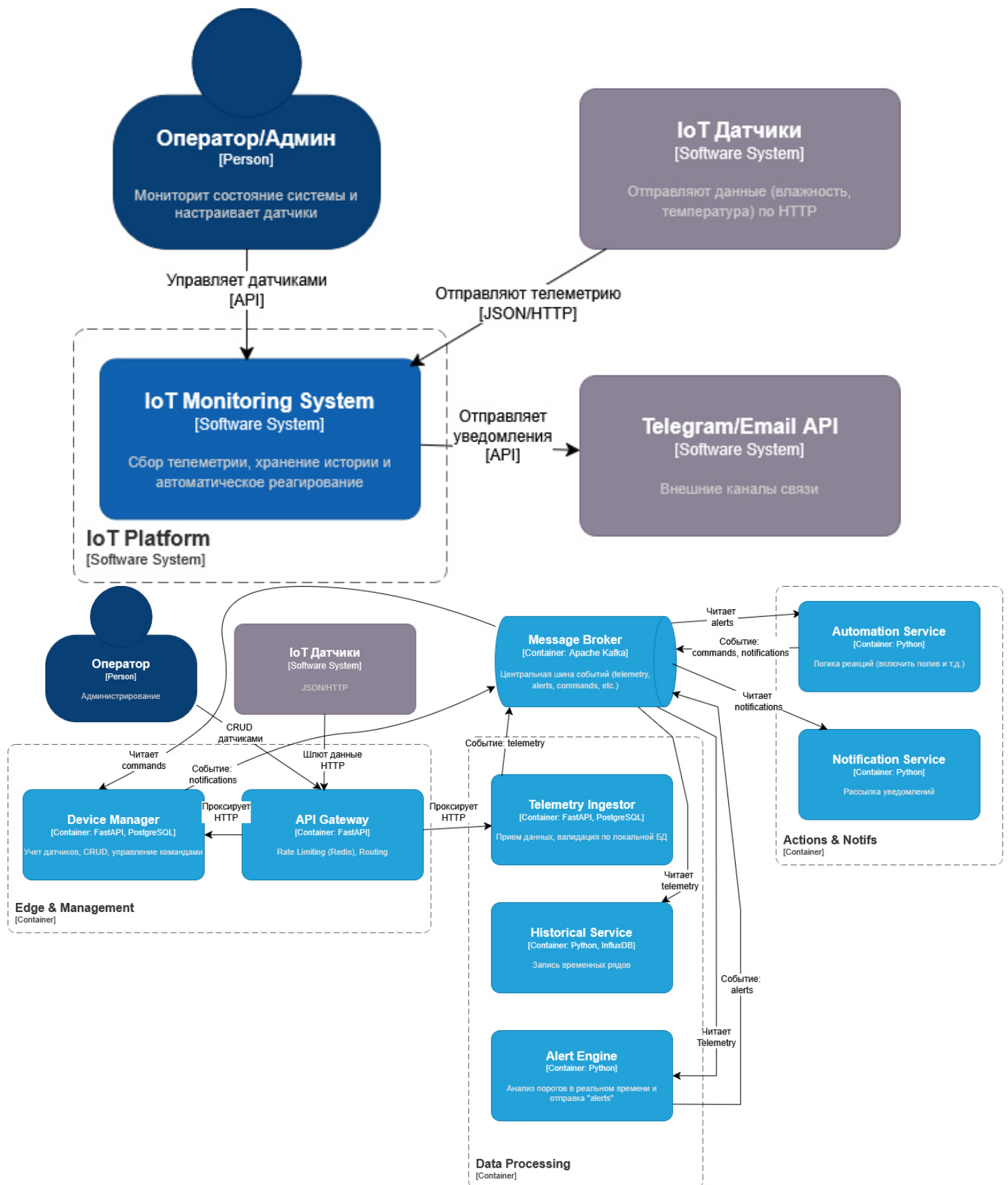
Use Cases

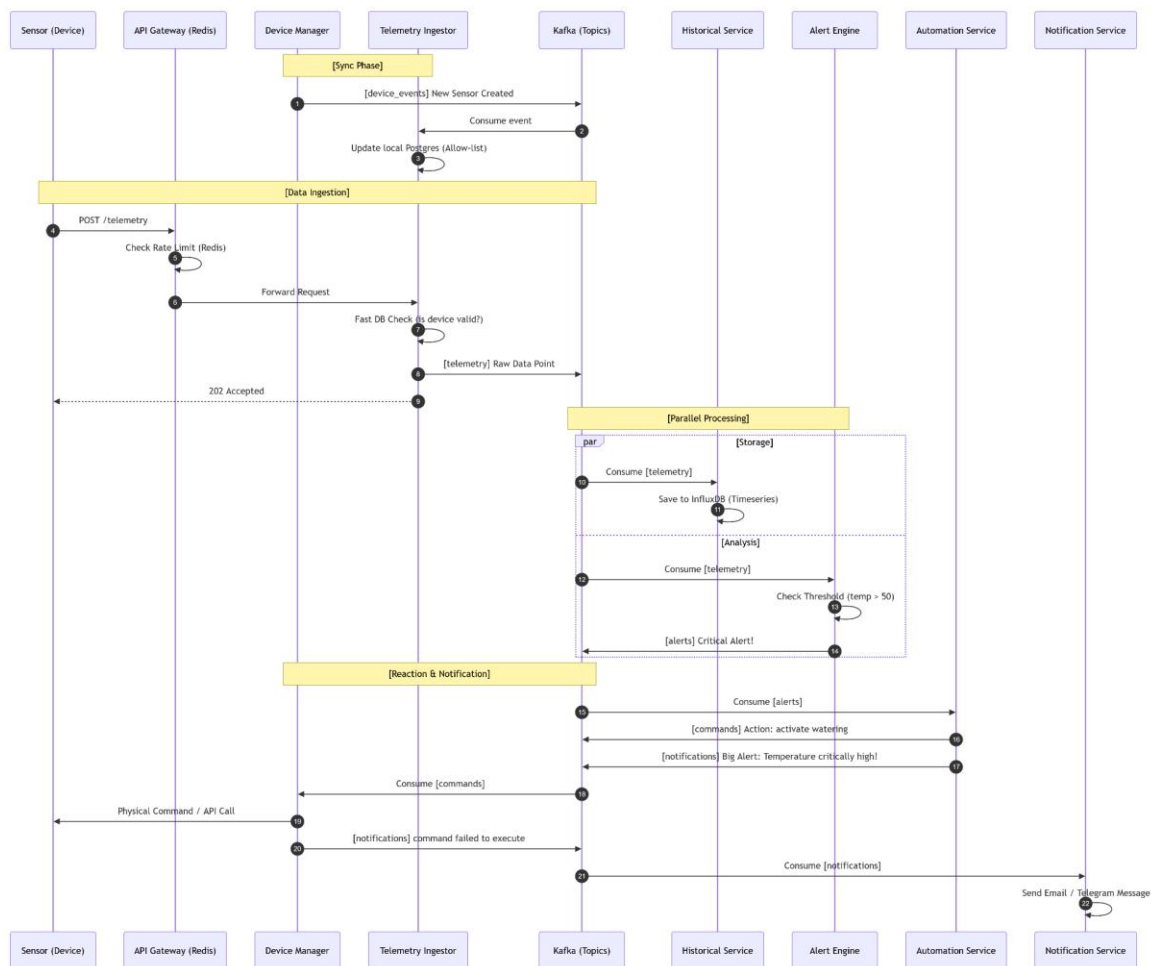
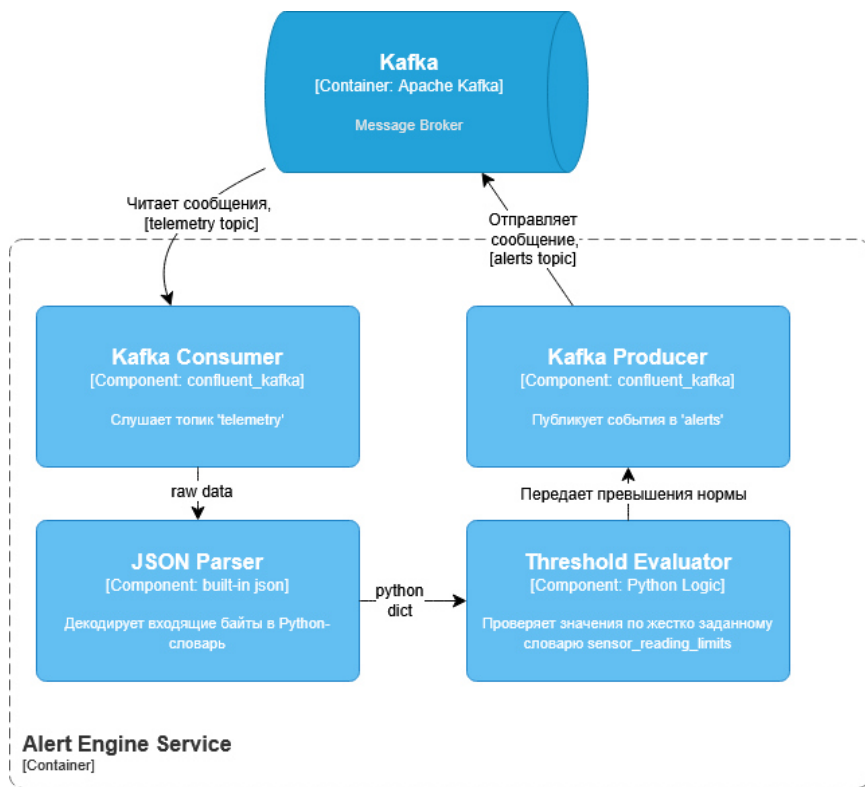
1. **UC-1: Регистрация и синхронизация.** Администратор создает датчик в **Device Manager**. Система отправляет событие в топик **device_events**. **Telemetry Ingestor** обновляет свою локальную БД, чтобы мгновенно узнавать этот датчик в будущем.
2. **UC-2: Прием данных с защитой.** Датчик шлет POST-запрос. Gateway проверяет лимиты в Redis. Если лимит превышен - возвращает 429. Если ок - **Ingestor** сверяет ID с базой и кладет данные в **telemetry**.
3. **UC-3: Алертинг и реакция.** **Alert Engine** выявляет превышение порога и создает событие в **alerts**. **Automation Service** перехватывает его, генерирует команду для **Device Manager** через топик **commands**.

NFR (Нефункциональные требования)

1. **Производительность:** Обработка события (от входа в Ingestor до выхода в Automation) - не более **2 сек.**
2. **Отказоустойчивость:** При падении **Historical Service** данные должны накапливаться в Kafka и обрабатываться после восстановления сервиса.
3. **Масштабируемость:** Система должна поддерживать добавление новых потребителей данных в Kafka без изменения кода отправителя.
4. **Безопасность:** Ограничение частоты запросов (Rate Limiting) - не более **100 запросов в секунду** на один ID датчика.

3. C4 (L1-3) + Sequence Diagram





Код для самостоятельного воспроизведения Sequence Diagram в Mermaid:

```
sequenceDiagram
    autonumber
    participant D as Sensor (Device)
    participant GW as API Gateway (Redis)
    participant DM as Device Manager
    participant ING as Telemetry Ingestor
    participant K as Kafka (Topics)
    participant HIST as Historical Service
    participant AE as Alert Engine
    participant AUTO as Automation Service
    participant NOTIF as Notification Service

    Note over DM, ING: [Sync Phase]
    DM->>K: [device_events] New Sensor Created
    K->>ING: Consume event
    ING->>ING: Update local Postgres (Allow-list)

    Note over D, K: [Data Ingestion]
    D->>GW: POST /telemetry
    GW->>GW: Check Rate Limit (Redis)
    GW->>ING: Forward Request
    ING->>ING: Fast DB Check (is device valid?)
    ING->>K: [telemetry] Raw Data Point
    ING-->>D: 202 Accepted

    Note over K, AE: [Parallel Processing]
    par Storage
        K->>HIST: Consume [telemetry]
        HIST->>HIST: Save to InfluxDB (Timeseries)
    and Analysis
        K->>AE: Consume [telemetry]
        AE->>AE: Check Threshold (temp > 50)
        AE->>K: [alerts] Critical Alert!
    end

    Note over K, DM: [Reaction & Notification]
    K->>AUTO: Consume [alerts]
    AUTO->>K: [commands] Action: activate watering
    AUTO->>K: [notifications] Big Alert: Temperature critically high!

    K->>DM: Consume [commands]
    DM->>D: Physical Command / API Call
    DM->>K: [notifications] command failed to execute

    K->>NOTIF: Consume [notifications]
    NOTIF->>NOTIF: Send Email / Telegram Message
```

4. Микросервисы.

API Gateway - единая точка входа для всех датчиков; фильтрует входящий трафик, ограничивает частоту запросов (Rate Limiting) через Redis и защищает систему от DDoS-атак [Быков Кирилл].

Device Manager - управляет реестром датчиков (CRUD) и координирует выполнение команд управления устройствами [Быков Кирилл].

Telemetry Ingestor - принимает данные от датчиков, проверяет их права на отправку и пересылает сообщения в Kafka [Киселев Егор].

Historical Data Service - сохраняет поток телеметрии в базу данных временных рядов (InfluxDB) и предоставляет API для отрисовки графиков [Киселев Егор].

Alert Engine - анализирует данные в реальном времени и генерирует события тревоги при нарушении заданных порогов [Гилязов Ренат].

Automation Service - исполняет логику реагирования (сценарии), формирует команды для устройств и инициирует алерты [Гилязов Ренат].

Notification Service - слушает топик уведомлений и отправляет сообщения конечным пользователям (Email/Telegram) [Киселев Егор, Гилязов Ренат].

5. Kafka, RabbitMQ, NATS

Взаимодействие между сервисами построено на принципе **Event-Driven Architecture (EDA)** - они не вызывают друг друга напрямую, а обмениваются сообщениями через топики.

- **Синхронизация данных:** **Device Manager** отправляет события (создание/удаление датчика) в топик **device_events**, а **Telemetry Ingestor** слушает их, чтобы обновлять свою локальную базу разрешенных устройств.
- **Поток телеметрии:** **Ingestor** отправляет сырые данные в топик **telemetry**. Оттуда их параллельно забирают **Historical Service** (для записи в БД) и **Alert Engine** (для анализа).
- **Цепочка реакции:** Если **Alert Engine** видит проблему, он пишет в топик **alerts**. Его слушает **Automation Service**, который принимает решение и отправляет команду в топик **commands** и критические “alerts” в топик **notifications**.
- **Исполнение и уведомление:** **Device Manager** забирает команды из **commands** для управления датчиком (в случае ошибки, возвращает сообщение в **notifications**), а **Notification Service** собирает сообщения из топика **notifications** от всех сервисов, чтобы отправить их пользователю.

Параметр	Kafka	NATS JetStream	RabbitMQ
Throughput	500k – 1M+ msg/sec	200k – 400k msg/sec	50k – 100k msg/sec
Latency	10–50 мс	0.5–5 мс	5–20+ мс
Durability & Retention	Отличная (distributed log, retention дни/недели)	Хорошая (streams)	Хорошая (quorum queues)
Replay событий	Идеальный (consumer offset)	Хороший	Ограниченный
Параллельные consumers	Consumer groups + partitioning	Pull/push consumers	Competing consumers
Ordering	Гарантировано в partition	Гарантировано в stream	Гарантировано в queue
Сложность	Выше	Низкая	Средняя
Лучше всего для	Высоконагруженная телеметрия, IoT pipelines, real-time analytics	Низкая latency, edge/IoT, простые микросервисы	Task queues, сложный routing, RPC

Наш проект – IoT–система с высоким потоком телеметрии, параллельной обработкой событий (storage + анализ) и цепочкой реакций (alert → command → notification).

Почему выбрали Kafka:

- Отлично справляется с большим объемом телеметрии
- Позволяет нескольким сервисам параллельно читать одни и те же события
- Обеспечивает надежное хранение + возможность replay (перечитать историю)
- Гарантирует порядок событий внутри устройства (partition по device_id)

NATS JetStream быстрее по latency и проще, но уступает в throughput и долговременной надежности при росте нагрузки. Подошел бы только при небольшом объеме данных (например, система автоматизации дома).

RabbitMQ удобен для очередей и сложного роутинга, но его пропускная способность в разы ниже - он стал бы bottleneck на нашем потоке телеметрии.

6. Данные

PostgreSQL (Реляционная БД) - используется в **Device Manager** и **Telemetry Ingestor** как надежное хранилище мастер-данных. Здесь лежат реестры датчиков, их ID и настройки.

Redis (In-memory БД) - выполняет две критические задачи:

1. **Rate Limiting:** Хранит счетчики запросов с IP-адресов в реальном времени, позволяя мгновенно блокировать «спам» от датчиков или операторов/админов системы мониторинга.
2. **Кэширование состояний:** Хранит актуальный статус доступности датчиков в **Telemetry Ingestor**, чтобы не перегружать основную PostgreSQL при проверке каждого входящего пакета.

InfluxDB (Time-series БД) - используется в **Historical Service**. В отличие от обычных баз, она оптимизирована специально под временные ряды. Это позволяет молниеносно записывать огромные потоки метрик и эффективно выгружать данные за определенные периоды для отрисовки графиков.

7. Кэширование Redis-like

Кэширование используется для сервиса **Telemetry Ingestor** с помощью Redis (сохраняется идентификатор девайса и отслеживается ли он, чтобы каждый раз не обращаться к PostgreSQL).

8. Слой роутинга: Маршрутизатор + Балансировщик + Rate Limiter

Маршрутизатор считывает название сервиса прямо из URL и мгновенно перенаправляет трафик на нужный внутренний адрес в сети Docker. Параллельно с этим **rate limiter** через специальную прослойку middleware проверяет каждый IP в базе Redis и блокирует пользователя ошибкой 429 при превышении лимита запросов.

Балансировка - примитивная, (Round Robin) на уровне DNS через Docker (docker-compose up -scale device_manager=3).

9. Зоны ответственности

Каждый из нас принимал участие при планировании проекта и решении вопросов возникающих по ходу его реализации (как сервисы будут общаться через Kafka, какой стек будет на проекте, какие-то уточнения по работе микросервисов и так далее), слой роутинга (Маршрутизатор + Балансировщик + Rate Limiter) тоже обсуждался и планировался вместе (реализовал Быков Кирилл 11-311), распределение по микросервисам продемонстрировано выше.

10. Инфраструктура Kubernetes

10.1 Локальный кластер Kubernetes с Cilium CNI

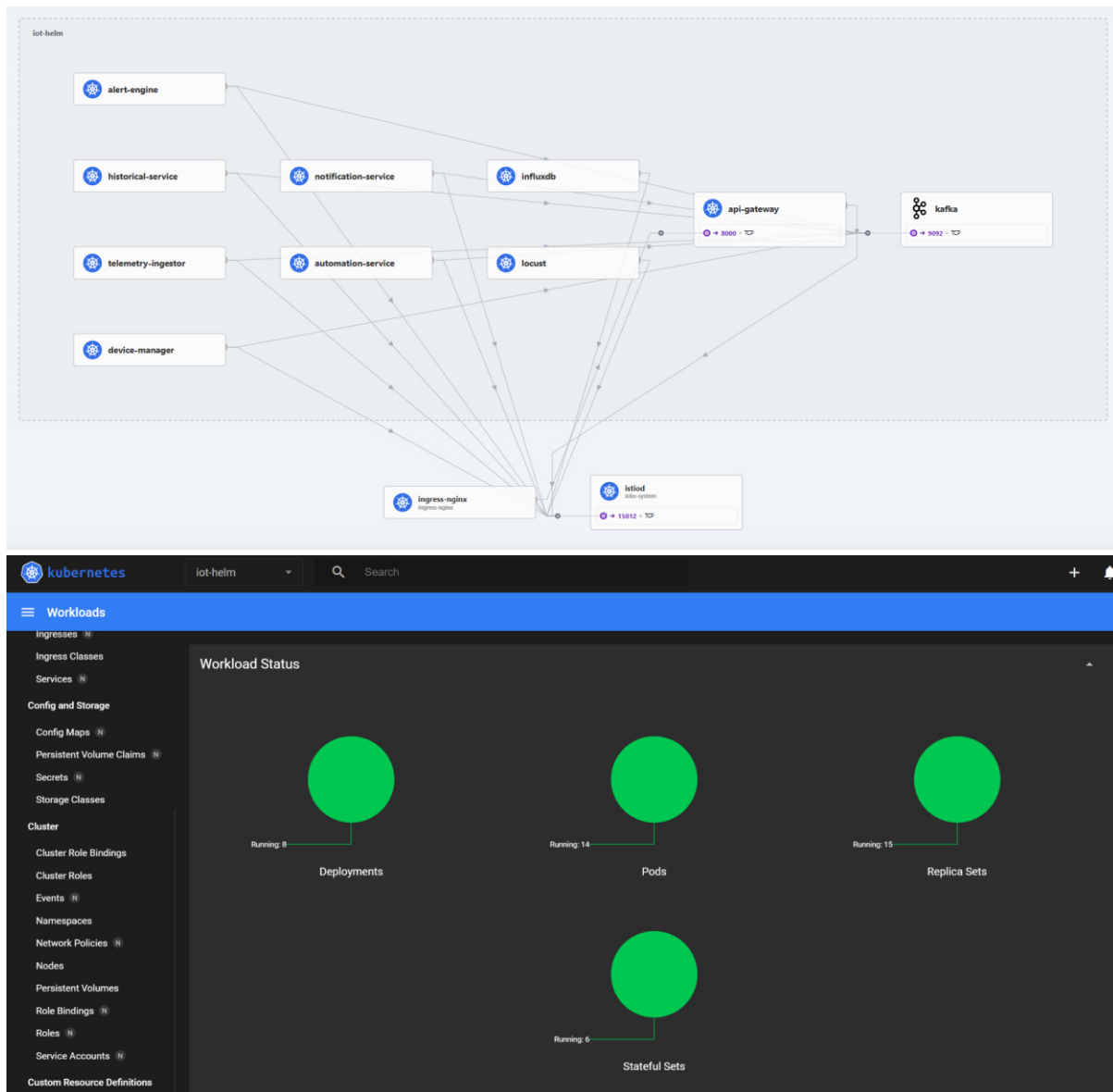
Для развертывания платформы был поднят локальный кластер Kubernetes на базе Minikube (driver=docker, 4 CPU, 6 GB RAM) с двумя нодами. В качестве CNI был установлен Cilium вместо стандартного kindnet, что обеспечивает eBPF-наблюдаемость и поддержку сетевых политик.

Конфигурация кластера:

- Minikube запущен с флагом --cni=false для подключения Cilium вместо стандартного сетевого плагина
- Добавлена вторая воркер-нода (minikube-m02) для распределения нагрузки
- Cilium установлен через Helm с параметром operator.replicas=1 (под одну control-plane ноду)
- Все 22 пода кластера управляются Cilium (Cluster Pods: 22/22 managed by Cilium)
- Hubble (UI для eBPF-наблюдаемости) включен для визуализации трафика между микросервисами в реальном времени

Преимущества Cilium перед стандартным CNI:

- Замена kube-proxy через eBPF - более быстрая обработка сетевых правил
- Сетевые политики - точный контроль взаимодействия между микросервисами
- Hubble UI - граф трафика между подами в реальном времени, удобный для отладки микросервисной архитектуры



10.2 Автоматическое масштабирование (HPA)

Для автоматического масштабирования воркер-нод был настроен Horizontal Pod Autoscaler (HPA). Karpenter и Cluster Autoscaler требуют облачного провайдера (AWS/GCP/Azure) для создания реальных VM-нод, поэтому для локального Minikube была применена альтернатива: HPA масштабирует поды между двумя нодами кластера при росте CPU-нагрузки.

Настройка HPA для микросервисов:

- **api-gateway:** min=1, max=3, порог CPU=50%
- **device-manager:** min=1, max=3, порог CPU=50%
- **telemetry-ingestor:** min=1, max=3, порог CPU=50%

Работоспособность была проверена нагрузочным тестом через Locust: при 10 пользователях CPU api-gateway вырос до 470%, НРА автоматически масштабировал деплоймент с 1 до 10 реплик (тест с php-apache), поды распределились между нодами minikube и minikube-m02.

10.3 IaC - Terraform

Базовая инфраструктура кластера описана в Terraform с использованием провайдера hashicorp/kubernetes. Terraform управляет следующими ресурсами:

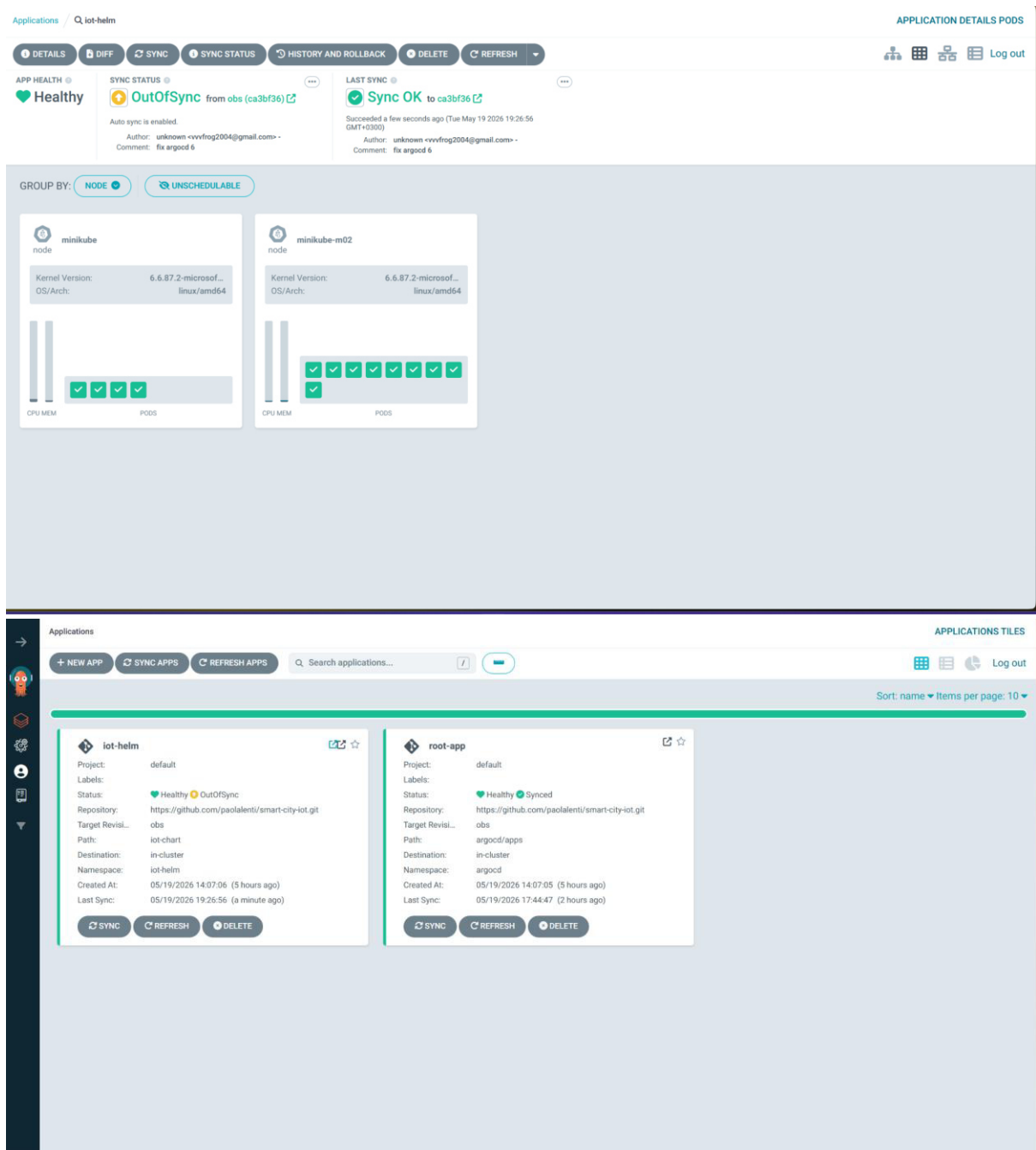
- **Namespaces:** iot-helm, vault, external-secrets, monitoring, argocd - все с меткой managed-by=terraform
- **ServiceAccounts:** admin-user (Kubernetes Dashboard), argocd-manager (ArgoCD)
- **ClusterRoleBindings:** cluster-admin для Dashboard и ArgoCD manager
- **ConfigMap vault-iot-policy:** политика доступа к секретам Vault
- **Secret iot-app-defaults:** базовые значения секретов приложения (заглушка, реальные значения в Vault)

10.4 GitOps - ArgoCD (App of Apps)

ArgoCD развернут в namespace argocd и настроен на автоматическую синхронизацию приложений из Git-репозитория по паттерну App of Apps:

- **root-app** - корневое приложение ArgoCD, следит за директорией argocd/apps в репозитории и автоматически регистрирует дочерние приложения
- **iot-helm** - дочернее приложение, деплоит Helm-чарт из пути iot-chart ветки obs(использовали для удобства разработки) репозитория <https://github.com/paolalenti/smart-city-iot.git> в namespace iot-helm

После подключения ArgoCD управление чартом перешло полностью под GitOps - изменения применяются через Git, а не через прямые вызовы helm upgrade. Это исключает расхождения между состоянием репозитория и кластера.



10.5 Ansible Role для Kafka

Для автоматизированного деплоя Kafka написана Ansible Role (коллекция kubernetes.core), запускаемая через Docker без локальной установки Ansible. Role выполняет следующие задачи:

- Применяет манифест StatefulSet и Service для Kafka из Jinja2-шаблона kafka.yml.j2
- Ожидает готовности пода kafka-0 (timeout 120 сек)
- Создает топики: telemetry, alerts, commands, notifications, device_events

- Выводит список всех существующих топиков для верификации

Запуск осуществляется через PowerShell-скрипт `run.ps1`, который автоматически генерирует `kubeconfig` с Linux-путями для работы внутри Docker-контейнера.

10.6 Управление секретами - Vault + External Secrets

HashiCorp Vault развернут изначально в dev-режиме в namespace `vault`, позже заменен на `Prod` (После перезапуска настройки не сбрасываются, нужно только произвести `unseal` хранилища). Секреты приложения хранятся по пути `iot-helm/app` в KV v2 движке. External Secrets Operator синхронизирует их в Kubernetes Secret `iot-secrets` с интервалом 15 секунд.

Секреты в Vault: `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_DB_TELEMETRY`, `INFLUXDB_URL`, `INFLUXDB_TOKEN`, `INFLUXDB_ORG`, `INFLUXDB_BUCKET`, `DOCKER_INFLUXDB_INIT_USERNAME`, `DOCKER_INFLUXDB_INIT_PASSWORD`, `BOT_TOKEN`, `CHAT_ID`.

Аутентификация External Secrets в Vault осуществляется через Kubernetes `auth` метод с ролью `iot-helm-role`, привязанной к ServiceAccount `vault-auth` в namespace `iot-helm`.

10.7 Нагрузочное тестирование - Locust

Locust развернут как Deployment внутри кластера Kubernetes в namespace `iot-helm`. Это позволяет генерировать нагрузку изнутри кластера, обращаясь к сервисам напрямую через Ingress controller по его ClusterIP, с передачей заголовка `Host: iot-helm.local`.

Сценарии тестирования: регистрация устройства (POST `/device_manager/devices/`), отправка телеметрии (POST `/telemetry/ingest/`) с весом 5, получение информации об устройстве (GET `/device_manager/devices/{id}/`) с весом 1. Каждый виртуальный пользователь генерирует случайный X-Forwarded-For заголовок для симуляции разных IP-адресов.

Результаты теста: при 10 пользователях достигнут RPS ~1.27, при 100 пользователях ~1.55 RPS с 33% ошибок типа 429 Too Many Requests - что подтверждает корректную работу Rate Limiting на уровне API Gateway.

10.8 Ingress + HAProxy + Keepalived

В качестве Ingress Controller используется nginx-ingress с TLS-терминацией (self-signed сертификат для домена `iot-helm.local`). Для обеспечения отказоустойчивости точки входа настроены HAProxy и Keepalived.

- **Nginx Ingress:** маршрутизирует входящий HTTPS-трафик на `api-gateway:8000`, тип сервиса переключен на LoadBalancer для прямого доступа по домену без проброски портов
- **HAProxy:** балансировщик нагрузки перед Ingress, обеспечивает распределение трафика и health-check бэкендов
- **Keepalived:** обеспечивает отказоустойчивость через виртуальный IP (VIP) - при падении основного узла трафик автоматически переходит на резервный

10.9 Service Mesh - Istio

Istio развернут в кластере для организации Service Mesh между микросервисами. Настроены следующие политики:

- **Mutual TLS (mTLS):** автоматические сертификаты от Istio для всех сервисов в mesh, весь трафик между микросервисами шифруется и аутентифицируется без изменений в коде приложений
- **Retry-политика:** настроена политика повтора запросов для случаев gateway error - при временной недоступности сервиса Istio автоматически повторяет запрос без участия клиента
- **Circuit Breaker (Outlier Detection):** настроена политика автоматического отключения неисправных подов из балансировки - при превышении порога ошибок под исключается из пула и трафик

перенаправляется на здоровые экземпляры

Namespaces			
Namespace ▾	Filter by Namespace		
Namespace ↑	Type ↑ ⓘ	Mode ↑	Health ↑ ⓘ
NS cilium-secrets	-	Not applicable	🔍 n/a
NS default	-	Not applicable	🔍 n/a
NS external-secrets	-	Not applicable	✅ Healthy
NS ingress-nginx	-	Not applicable	✅ Healthy
NS iot-helm	DP	Sidecar	✅ Healthy
NS istio-system	CP	Sidecar	✅ Healthy
NS kubernetes-dashboard	-	Not applicable	✅ Healthy
NS monitoring	-	Not applicable	✅ Healthy
NS vault	-	Not applicable	✅ Healthy

10.10 Observability - Prometheus, Grafana, Loki, Jaeger

Развернут полный стек наблюдаемости в namespace monitoring:

- **Prometheus + Grafana:** сбор метрик со всех подов кластера, предустановленные дашборды для мониторинга латентности, CPU/Memory, метрик Kafka и срабатываний Rate Limiter
- **Loki + Promtail:** централизованный сбор логов всех подов namespace iot-helm, доступны через Grafana Explore с фильтрацией по сервисам
- **Jaeger:** distributed tracing, установлен и подключен как datasource в Grafana для трассировки запросов между микросервисами



Alertmanager

Alertmanager | <http://prometheus-kube-prometheus-alertmanager.monitoring:9093/>

Build a dashboard

Explore



Jaeger

Jaeger | <http://jaeger:16686>

Build a dashboard

Explore



Loki

Loki | <http://loki:3100>

Build a dashboard

Explore



Prometheus

Prometheus | <http://prometheus-kube-prometheus-prometheus.monitoring:9090/>

Build a dashboard

Explore

Loki

Go queryless

Split

Add

Last 1 hour

Run query

←

Logs

1000 lines shown - 16.16% (9min 42sec) of 1h Total bytes processed: 462 kB Common labels: namespace=iot-helm node_name=minikube

Search fields by name

Show log level

Fields

app

100%

component

4%

container

100%

filename

100%

instance

4%

job

100%

namespace

100%

node_name

100%

pod

100%

2026-05-19 18:52:15.390 INFO {

"log": "[2026-05-19 15:52:15.352] INFO [NodeToControllerChannelManager id=1 name=registration] Node 1 disconnected. (org.apache.kafka.clients.NetworkClient)\n",

"stream": "stdout",

"time": "2026-05-19T15:52:15.352836725Z"

}

2026-05-19 18:47:52.916 {

"log": "2026-05-19 15:47:52.788 GMT [94] LOG: checkpoint complete: wrote 6 buffers (0.0%), wrote 0 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.602 s, sync=0.002 s, total=0.612 s; sync files=3, longest=0.001 s, average=0.001 s; distance=42 kB, estimate=273 kB; lsn=0/1C1C688, redo lsn=0/1C1C630\n",

"stream": "stderr",

"time": "2026-05-19T15:47:52.788732638Z"

}

2026-05-19 18:47:52.416 {

"log": "2026-05-19 15:47:52.176 GMT [94] LOG: checkpoint starting: time\n",

"stream": "stderr",

"time": "2026-05-19T15:47:52.176714341Z"

}

2026-05-19 18:46:43.648 {

"log": "2026-05-19 15:46:43.536 GMT [98] LOG: checkpoint complete: wrote 72 buffers (0.4%), wrote 3 SLRU buffers; 1 WAL file(s) added, 0 removed, 0 recycled; write=7.227 s, sync=0.268 s, total=7.686 s; sync files=60, longest=0.066 s, average=0.005 s; distance=296 kB, estimate=296 kB; lsn=0/1C18740, redo lsn=0/1C186B0\n",

"stream": "stderr",

"time": "2026-05-19T15:46:43.536407042Z"

}

2026-05-19 18:46:35.998 {



10.11 CI/CD - GitHub Actions + Kaniko

Настроен CI/CD пайплайн на базе GitHub Actions с Self-Hosted Runner в namespace github-runner внутри кластера Kubernetes.

Этапы пайплайна:

- Подготовка: Исходный код через helper-под копируется в PVC (kaniko-context) для поочередной сборки сервисов.
- Build (Kaniko): Сборка Docker-образов без Docker daemon прямо в подах Kubernetes. Образы тегируются по Git SHA.
- Push в Registry: Публикация артефактов во внутрикластерный registry.kube-system.svc.cluster.local:80.

- Update Config: Пайплайн обновляет новые теги в values.yaml и коммитит изменения в ветку obs.
- Deploy & Sync: Вызывается прямой helm upgrade, после чего ArgoCD автоматически синхронизирует Helm-чарт с кластером.

Единый Helm-чарт покрывает все 7 микросервисов платформы с настройками подключения к Kafka, Redis, PostgreSQL и InfluxDB.

11. Зоны ответственности по инфраструктуре

Распределение задач инфраструктурной части проекта между участниками команды:

Киселев Егор - Kubernetes и развертывание

- Первоначальное развертывание кластера Kubernetes (Minikube)
- Написание и настройка Helm-чартов для всех микросервисов платформы
- Интеграция HashiCorp Vault + External Secrets Operator
- Установка и настройка Cilium CNI с Hubble
- Настройка Ingress Controller и HPA (автоскейлинг)
- Объединение веток и итоговое развертывание всех компонентов в единый кластер

Быков Кирилл - Observability и CI/CD

- Развертывание стека Observability: Prometheus, Grafana, Loki + Promtail, Jaeger
- Настройка ArgoCD (GitOps, App of Apps паттерн)
- Настройка CI/CD пайплайна: GitHub Actions Self-Hosted Runner, сборка через Kaniko
- IaC - описание инфраструктуры в Terraform (namespaces, ServiceAccounts, RBAC)
- Ansible Role для автоматизированного деплоя Kafka и создания топиков

Гилязов Ренат - Service Mesh и трафик

- Развертывание Istio и настройка Service Mesh

- Настройка mutual TLS с автоматическими сертификатами Istio для всех сервисов
- Настройка retry-политик для обработки gateway error между микросервисами
- Настройка Circuit Breaker (Outlier Detection) - автоматическое отключение неисправных подов из балансировки
- Настройка HAProxy + Keepalived для балансировки и отказоустойчивости точки входа
- Нагрузочное тестирование Locust и валидация Circuit Breaker при искусственном отказе сервисов